APACHE KAFKA TRAINING PROGRAM

SECURITY BASICS

ACLs & Authorization

Study Material

Author: Rajesh Pasham

Apache Kafka Training Program | Security Module | 2026

Module Overview

This module covers Kafka's built-in authorization framework — Access Control Lists (ACLs). You will learn how Kafka enforces who can do what on which resources, how to configure and manage ACLs using CLI tools and the AdminClient API, and how to troubleshoot common authorization failures in production.

Learning Objectives

- Understand Kafka's security architecture and the role of authorization
- Differentiate between Authentication and Authorization in Kafka
- Understand the Kafka ACL data model: Principal, Resource, Operation, Permission
- Configure kafka.authorizer.class and related broker settings
- Create, list, describe, and delete ACLs using kafka-acls.sh
- Implement ACLs for Producers, Consumers, and Kafka Connect
- Understand Wildcard ACLs, Resource Patterns, and PREFIXED resources
- Troubleshoot common ACL-related authorization errors from broker logs
- Apply ACL best practices for multi-tenant Kafka environments

Prerequisites

- Completion of Day 1 (Kafka Fundamentals) and Day 2 (Administration)
- Understanding of Kafka Topics, Producer, Consumer, Consumer Groups
- Basic familiarity with SSL/SASL concepts (covered in Advanced Security module)

Module 1: Kafka Security Architecture

Kafka's security model is built on three independent pillars that work together to protect your cluster. Understanding how they relate is essential before diving into ACLs.

1.1 The Three Security Pillars

Pillar	Question It Answers	Kafka Mechanism
Encryption	Can the data be read in transit?	TLS/SSL (wire encryption)
Authentication	Who is this client?	SASL (PLAIN, SCRAM, GSSAPI/Kerberos), mTLS
Authorization	Is this client allowed to do this action?	ACLs (this module)

Key Insight: Authentication and Authorization are separate concerns. Authentication proves identity; Authorization decides what that identity is allowed to do. ACLs only work meaningfully when authentication is also enabled — otherwise every client is treated as ANONYMOUS.

1.2 Kafka's Default Security Posture

By default, Kafka ships with NO security enabled:

- No encryption — all data flows in plaintext
- No authentication — any client can connect
- No authorization — any connected client can read/write any topic

WARNING: The default open configuration is acceptable only for development. Any production cluster must have at minimum authentication + ACLs enabled to prevent unauthorized access and data exfiltration.

1.3 How Kafka Processes a Client Request

When a client (producer or consumer) sends a request to a broker, Kafka processes it in this sequence:

1. Client connects via network listener (PLAINTEXT or SSL port)
2. Broker authenticates the client using the configured SASL mechanism or mTLS certificate
3. Broker identifies the client's Principal (e.g., User:alice or CN=service-account)
4. Broker checks if an ACL exists for: [Principal] + [Operation] + [Resource]
5. If ACL permits: request proceeds. If no ACL or denied: AUTHORIZATION_FAILED error returned

The authorizer is invoked after authentication. If allow.everyone.if.no.acl.defined=true (default when no authorizer is configured), all requests pass. Once an authorizer is configured, this defaults to false and all access must be explicitly granted.

Module 2: The ACL Data Model

A Kafka ACL is a rule that grants or denies a specific Principal the ability to perform a specific Operation on a specific Resource. Understanding each component is critical before writing ACL commands.

2.1 Principal

A Principal represents the identity of the client as authenticated by Kafka. The format is:

```
Principal-Type:Principal-Name
```

Common Principal Types

Authentication Method	Principal Format	Example
SASL/PLAIN or SCRAM	User:<username>	User:alice
Kerberos (GSSAPI)	User:<kerberos-principal>	User:kafka-producer@REALM.COM
mTLS (client certificate)	User:<CN from cert>	User:CN=data-pipeline,OU=Engineering
No authentication	User:ANONYMOUS	User:ANONYMOUS

Note: The principal name is case-sensitive and must exactly match the authenticated identity. User:Alice and User:alice are different principals.

2.2 Resource Types

ACLs can be applied to the following Kafka resource types:

Resource Type	Description	CLI Flag
Topic	A Kafka topic	--topic <name>
Group	A consumer group	--group <name>
Cluster	The Kafka cluster itself (admin operations)	--cluster
TransactionalId	Transactional producer identity	--transactional-id <id>
DelegationToken	Delegation token (token-based authentication)	--delegation-token <token>

2.3 Operations

Operations define what action the principal is attempting. Each resource type supports a specific set of operations:

Topic Operations

Operation	What It Allows	Needed By
Read	Consumer reading messages from a topic	Consumers
Write	Producer writing messages to a topic	Producers
Create	Creating a new topic	Admin, auto-create producers
Delete	Deleting a topic	Admins only
Alter	Modifying topic configuration	Admins only
Describe	Viewing topic metadata, offsets, configs	All clients (often needed implicitly)
AlterConfigs	Changing topic-level configs via AdminClient	Admins only
DescribeConfigs	Reading topic-level configs	Monitoring tools, admins

Group Operations

Operation	What It Allows	Needed By
Read	Joining a consumer group, committing offsets	Consumers
Describe	Viewing consumer group metadata and lag	Monitoring tools, admins, consumers
Delete	Deleting a consumer group	Admins only

Cluster Operations

Operation	What It Allows	Needed By
Create	Creating topics (cluster-level alternative to Topic:Create)	Admins
Alter	Modifying broker configs, reassigning partitions	Admins
Describe	Viewing broker metadata, cluster ID	All clients
ClusterAction	Inter-broker replication, controller actions	Brokers (internal)
AlterConfigs	Dynamically modifying broker configurations	Admins
DescribeConfigs	Reading broker-level configurations	Monitoring tools

Operation	What It Allows	Needed By
IdempotentWrite	Idempotent producer operations (acks=-1 deduplication)	Idempotent producers

2.4 Permission Types

Permission	Effect	Use Case
ALLOW	Explicitly grants the operation to the principal	Standard access grants
DENY	Explicitly denies the operation to the principal — overrides any ALLOW	Blacklisting a specific user from a sensitive topic

DENY Precedence Rule: DENY always wins over ALLOW. If a principal has both an ALLOW ACL and a DENY ACL for the same operation+resource, the DENY takes effect. Use DENY sparingly — it can cause hard-to-debug authorization failures.

Module 3: Configuring the Kafka Authorizer

3.1 Enabling Authorization on the Broker

To enable ACL-based authorization, add the following to each broker's server.properties:

```
# For Kafka with ZooKeeper (Kafka <= 3.4 without KRaft)
authorizer.class.name=kafka.security.authorizer.AclAuthorizer
```

```
# For Kafka with KRaft mode (Kafka 3.3+)
authorizer.class.name=org.apache.kafka.metadata.authorizer.StandardAuthorizer
```

Additional Required Settings

Property	Recommended Value	Purpose
allow.everyone.if.no.acl.defined	false	Deny all access unless explicitly granted by an ACL
super.users	User:kafka-admin;User:broker-internal	These users bypass all ACL checks (use carefully)
authorizer.class.name	(see above)	Selects the ACL implementation

Super Users: Super users are specified as a semicolon-delimited list in super.users. They completely bypass all ACL checks. Only add the broker's own internal identity and the main admin user here. Never add application service accounts.

3.2 Where ACLs Are Stored

ZooKeeper-based clusters (AclAuthorizer)

ACLs are stored in ZooKeeper under the /kafka-acl/ znode. The authorizer caches them in memory and watches ZooKeeper for changes. ACL updates take effect within seconds cluster-wide.

KRaft-mode clusters (StandardAuthorizer)

ACLs are stored in the Kafka metadata log itself — no ZooKeeper dependency. This makes ACL management more reliable and consistent with the KRaft architecture. ACL updates are propagated via the metadata quorum.

```
# Verify ACL storage in ZooKeeper (if applicable)
zookeeper-shell.sh localhost:2181 ls /kafka-acl
```

```
zookeeper-shell.sh localhost:2181 ls /kafka-acl/Topic
```

3.3 Listener Configuration for Security

Kafka brokers can have multiple listeners on different ports with different security protocols. A common production setup:

```
# In server.properties
listeners=PLAINTEXT://0.0.0.0:9092,SASL_SSL://0.0.0.0:9093
advertised.listeners=PLAINTEXT://broker-host:9092,SASL_SSL://broker-host:9093
security.inter.broker.protocol=SASL_SSL
```

> *Best Practice: Disable the PLAINTEXT listener in production or restrict it to localhost only. All external clients should connect via SASL_SSL. The PLAINTEXT port should never be exposed on a production cluster.*

Module 4: Managing ACLs with kafka-acls.sh

The kafka-acls.sh script is the primary CLI tool for managing Kafka ACLs. It communicates with the Kafka broker directly (for KRaft) or via ZooKeeper (for ZooKeeper-based clusters).

4.1 Command Syntax

```
kafka-acls.sh --bootstrap-server <broker:port> \
  [--add | --remove | --list] \
  --allow-principal User:<name>  OR  --deny-principal User:<name> \
  --operation <operation> \
  [--topic <name> | --group <name> | --cluster | --transactional-id <id>] \
  [--resource-pattern-type <LITERAL|PREFIXED>]
```

4.2 Creating ACLs — Producers

A Kafka producer needs Write and Describe permissions on the target topic. If the topic does not exist and auto-creation is enabled, it also needs Create on Cluster or the specific topic:

```
# Grant producer access to User:data-pipeline on topic 'orders'
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:data-pipeline \
  --operation Write \
  --operation Describe \
  --topic orders
```

Tip: Always grant both Write AND Describe to a producer. Without Describe, the producer cannot fetch topic metadata and will fail with TOPIC_AUTHORIZATION_FAILED before it even sends a message.

4.3 Creating ACLs — Consumers

A consumer needs Read + Describe on the topic, and Read on the consumer group:

```
# Grant consumer access: topic 'orders' + consumer group 'order-processors'
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:order-service \
  --operation Read \
  --operation Describe \
  --topic orders
```

```
kafka-acls.sh --bootstrap-server localhost:9092 \
```

```
  --add \
  --allow-principal User:order-service \
  --operation Read \
  --group order-processors
```

4.4 Creating ACLs — Admin Operations

```
# Grant cluster-level admin rights to User:kafka-admin
kafka-acls.sh --bootstrap-server localhost:9092 \
  --add \
  --allow-principal User:kafka-admin \
  --operation All \
  --cluster
```

WARNING: The 'All' operation grants every possible operation on the resource. Use it ONLY for dedicated admin accounts, never for application service accounts.

4.5 Listing ACLs

```
# List ALL ACLs in the cluster
kafka-acls.sh --bootstrap-server localhost:9092 --list
```

```
# List ACLs for a specific topic
kafka-acls.sh --bootstrap-server localhost:9092 --list --topic orders
```

```
# List all ACLs for a specific principal
kafka-acls.sh --bootstrap-server localhost:9092 --list \
  --allow-principal User:data-pipeline
```

4.6 Removing ACLs

```
# Remove Write permission for User:data-pipeline on topic 'orders'
kafka-acls.sh --bootstrap-server localhost:9092 \
  --remove \
  --allow-principal User:data-pipeline \
  --operation Write \
  --topic orders
```

Warning: There is no --remove-all shortcut. You must specify the exact principal, operation, and resource to remove an ACL. The --remove command will prompt for confirmation before deleting.

Module 5: ACLs for Common Kafka Components

5.1 Complete Producer ACL Set

Resource	Operation	Reason
Topic:<topic-name>	Write	Send messages to the topic
Topic:<topic-name>	Describe	Fetch topic metadata (required for connection)
Cluster	IdempotentWrite	Required only for idempotent/transactional producers (enable.idempotence=true)
Cluster	Describe	Required for initial metadata bootstrap

5.2 Complete Consumer ACL Set

Resource	Operation	Reason
Topic:<topic-name>	Read	Read messages from the topic
Topic:<topic-name>	Describe	Fetch offset and partition metadata
Group:<group-id>	Read	Join the consumer group, commit offsets
Group:<group-id>	Describe	Check group status (for monitoring tools)
Cluster	Describe	Bootstrap metadata (auto-negotiated at connect time)

5.3 Kafka Connect Worker ACLs

Kafka Connect workers need broad permissions because they act as both producers (writing to sink topics) and consumers (reading from source topics), plus they manage internal topics:

Resource	Operation(s)	Purpose
Topic:connect-configs	Read, Write, Describe, Create	Internal connector config topic
Topic:connect-offsets	Read, Write, Describe, Create	Connector offset storage topic
Topic:connect-status	Read, Write, Describe, Create	Connector status topic
Topic:<source-topics>	Read, Describe	Source connector: read from source topics

Resource	Operation(s)	Purpose
Topic:<sink-topics>	Write, Describe, Create	Sink connector: write to sink topics
Group:connect-*	Read, Describe	Connect worker consumer groups
Cluster	Create, Describe	Create internal topics, bootstrap metadata

5.4 Kafka Streams ACLs

Kafka Streams applications consume from input topics, write to output topics, and manage internal state store topics (named <application-id>-*):

Resource	Operation(s)	Purpose
Topic:<input-topics>	Read, Describe	Read from source topics
Topic:<output-topics>	Write, Describe, Create	Write to result topics
Topic:<app-id>-*	Read, Write, Describe, Create, Delete	Internal state store changelog topics
Group:<app-id>	Read, Describe	Application consumer group
Cluster	Describe, Create	Metadata and topic creation

Best Practice: Use Prefixed ACLs (covered in Module 6) for Kafka Streams internal topics. Grant Topic:<app-id>- with resource-pattern-type PREFIXED to cover all internal topics with a single ACL.

Module 6: Resource Patterns

Kafka ACLs support three resource pattern types that provide flexibility in how broadly or narrowly ACL rules are scoped.

6.1 LITERAL Pattern (Default)

An ACL with LITERAL pattern applies to exactly the named resource — no wildcards.

```
# LITERAL ACL — applies to exactly 'orders' topic only
kafka-acls.sh --bootstrap-server localhost:9092 --add \
  --allow-principal User:alice --operation Write \
  --topic orders --resource-pattern-type LITERAL
```

6.2 PREFIXED Pattern

A PREFIXED ACL applies to all resources whose name starts with the given prefix. This is extremely useful for Kafka Streams internal topics and multi-tenant environments:

```
# PREFIXED ACL — applies to ALL topics starting with 'payment-'
# e.g. payment-orders, payment-refunds, payment-notifications
kafka-acls.sh --bootstrap-server localhost:9092 --add \
  --allow-principal User:payments-service \
  --operation Read --operation Write --operation Describe \
  --topic payment- --resource-pattern-type PREFIXED
```

> *Example: A Kafka Streams app with application.id=order-processor would need access to all topics starting with 'order-processor-'. A single PREFIXED ACL handles all internal state store topics: --topic order-processor- --resource-pattern-type PREFIXED*

6.3 Wildcard ACL (*)

Setting the resource name to '*' with LITERAL pattern grants the operation on ALL resources of that type:

```
# Wildcard ACL — applies to ALL topics
kafka-acls.sh --bootstrap-server localhost:9092 --add \
  --allow-principal User:monitoring-service \
  --operation Describe \
  --topic '*'
```

WARNING — Wildcard ACLs: Wildcard (*) ACLs are very powerful and can be dangerous. Use them only for read-only monitoring/admin accounts. Never grant Write or Delete with a wildcard ACL in production. Prefer PREFIXED over wildcard wherever possible.

6.4 Pattern Precedence

When multiple ACL patterns match a request, Kafka evaluates them in this precedence order:

Priority	Pattern Type	Example	Notes
1 (Highest)	LITERAL (exact match)	Topic:orders	Most specific — always checked first
2	PREFIXED (longest match)	Topic:order- (matches order-service, order-events)	Longer prefixes take priority over shorter ones
3 (Lowest)	WILDCARD (*)	Topic:* (matches all topics)	Least specific — checked last

DENY Override: Regardless of pattern precedence, a DENY ACL always overrides an ALLOW ACL. Even if a LITERAL ALLOW exists, a WILDCARD DENY will block the request.

Module 7: Troubleshooting Authorization Failures

Authorization failures are a common operational issue when ACLs are enabled. This module provides a structured approach to diagnosing and resolving them quickly.

7.1 Client-Side Error Signatures

Error Message	Root Cause	Likely Missing ACL
TOPIC_AUTHORIZATION_FAILED	No Read/Write ACL on the topic	Topic:<name> Read or Write
GROUP_AUTHORIZATION_FAILED	No Read ACL on the consumer group	Group:<group-id> Read
CLUSTER_AUTHORIZATION_FAILED	No Cluster-level ACL for admin operation	Cluster Describe or Alter
TRANSACTIONAL_ID_AUTHORIZATION_FAILED	Missing ACL on transactional ID	TransactionalId:<id> Write
UNKNOWN_TOPIC_OR_PARTITION	Topic doesn't exist OR no Describe ACL	Topic:<name> Describe

7.2 Broker-Side Log Analysis

When authorization fails, the Kafka broker logs the denial. Check kafka-authorizer.log:

```
# Find all authorization failures
grep 'DENIED\|AuthorizationException\|is not Allowed' /opt/kafka/logs/kafka-
authorizer.log | tail -30
```

A typical denied ACL log entry looks like:

```
[INFO]  Principal = User:data-pipeline is Denied Operation = Write
        from host = 10.0.1.50 on resource = Topic:LITERAL:orders
```

This log entry tells you everything you need to write the fix:

- Principal: User:data-pipeline
- Operation: Write
- Resource: Topic orders
- Pattern: LITERAL

```
# Fix: Add the missing ACL
kafka-acls.sh --bootstrap-server localhost:9092 --add \
```

```
  --allow-principal User:data-pipeline --operation Write --topic orders
```

7.3 Systematic Diagnostic Workflow

6. Identify the principal — what user/service account is the client using?
   - Check client configuration: sasl.jaas.config, ssl.keystore, or PLAIN username
   - If unknown, check broker logs for the connecting identity

7. Reproduce the exact error — note the resource and operation from the error
   - TOPIC_AUTHORIZATION_FAILED on produce = missing Topic:Write
   - TOPIC_AUTHORIZATION_FAILED on consume = missing Topic:Read or Group:Read

8. List existing ACLs for the principal

```
kafka-acls.sh --bootstrap-server localhost:9092 --list \
  --allow-principal User:<principal-name>
```

9. Compare against the required ACL set for the operation (see Module 5 tables)
10. Add missing ACLs and retry
11. Check kafka-authorizer.log to confirm the DENIED entry disappears

7.4 Common Gotchas

Gotcha	Description	Fix
Case-sensitive principal	User:Alice != User:alice	Match exact case from authentication config
Wildcard not matching	Topic:* requires --resource-pattern-type LITERAL	Default LITERAL type with name '*' is wildcard
Missing Describe for producer	Producer fails before sending — no metadata access	Always add Describe alongside Write
Missing Group ACL for consumer	Consumer joins group but cannot commit offsets	Add Group:<group-id> Read ACL
DENY shadowing ALLOW	A broad DENY ACL blocks a specific ALLOW	Remove conflicting DENY or narrow its scope
Super user not working	super.users format uses semicolons not commas	Use: User:admin;User:broker — not commas
ACL not taking effect	In ZK-mode, cache refresh takes up to 30 seconds	Wait or restart broker; KRaft is near-instant

Module 8: ACL Best Practices

8.1 Principle of Least Privilege

Each service account should have only the minimum permissions required to function. Follow this discipline:

- Never use 'All' operation for application service accounts
- Never grant Delete unless the service specifically needs to delete topics
- Grant Describe only where needed (though it is low-risk, keep it intentional)
- Create separate service accounts for producers, consumers, and admin tools
- Never share service account credentials between different applications

8.2 Naming Conventions

Establish consistent naming conventions for principals and ACL-related resources:

Component	Naming Convention	Example
Service accounts	team-service-env	payments-producer-prod
Consumer groups	team-service-purpose-env	payments-order-listener-prod
Topic prefixes	team.domain.topic	payments.orders.created
Admin accounts	kafka-admin-env	kafka-admin-prod

8.3 ACL Lifecycle Management

- Store all ACL definitions in source control (Git) as infrastructure-as-code
- Use automation tools (Terraform kafka provider, Confluent Ansible) to manage ACLs declaratively
- Audit ACLs regularly — remove ACLs for decommissioned services
- Use PREFIXED ACLs for Kafka Streams and Connect to reduce ACL sprawl
- Never manually edit ZooKeeper ACL znodes — always use kafka-acls.sh or AdminClient

8.4 Multi-Tenant Kafka ACL Strategy

For shared Kafka clusters serving multiple teams, use topic prefixes per team:

```
# Team A owns all topics starting with 'team-a-'
kafka-acls.sh --bootstrap-server localhost:9092 --add \
  --allow-principal User:team-a-producer \
```

```
  --operation Write --operation Describe --operation Create \
  --topic team-a- --resource-pattern-type PREFIXED
```

```
# Team B cannot touch Team A's topics (no ACL = no access)
# If using allow.everyone.if.no.acl.defined=false, this is enforced automatically
```

8.5 ACL Audit Checklist

Audit Item	Frequency	Tool/Command
List all ACLs and review for stale entries	Monthly	kafka-acls.sh --list
Check kafka-authorizer.log for repeated DENIED	Weekly	grep DENIED kafka-authorizer.log
Verify super.users list is minimal	Quarterly	Review server.properties
Confirm no wildcard Write/Delete ACLs exist	Monthly	kafka-acls.sh --list | grep Write
Review ACLs for departing team members	On offboarding	kafka-acls.sh --remove

Module Summary & Key Takeaways

Core Concepts Recap

Concept	Key Point
ACL Structure	Principal + Resource + Operation + Permission (Allow/Deny)
Principal Format	User:<name> — must match authenticated identity exactly, case-sensitive
Authorizer Config	authorizer.class.name + allow.everyone.if.no.acl.defined=false
Producer Minimum ACLs	Topic:Write + Topic:Describe (+ Cluster:IdempotentWrite if idempotent)
Consumer Minimum ACLs	Topic:Read + Topic:Describe + Group:Read
DENY Behavior	DENY always overrides ALLOW regardless of specificity
PREFIXED Pattern	Best practice for Kafka Streams, Connect, and multi-tenant setups
Troubleshooting	Check kafka-authorizer.log for DENIED entries — logs tell you exactly what ACL is missing

> *Next Topic: Advanced Security — TLS/mTLS, Kerberos authentication, and SASL mechanisms. ACLs work in conjunction with these authentication methods to provide complete end-to-end Kafka security.*

Prepared by: Rajesh Pasham
Apache Kafka Training Program | Security Module | 2026